Where a CDCL SAT Solver Spends Its Time: Profiling, a Cycle-Level Cost Model, and a Controlled Comparison with MiniSat

Alperen Buğra Başar
Bilkent University
bugra.basar@ug.bilkent.edu.tr

October 2026

Abstract

I study the performance of a CDCL SAT solver written from scratch in C++17. A sampling profile places half of its run time in one search loop, and four per-operation costs, fitted over 39 runs, predict its total cycles with a median error of 2.5%. A memory experiment that holds the search fixed while growing the footprint 20 times locates the cost of leaving the L2 cache. A single run suggested the solver was 12 times faster than MiniSat; shuffled copies and removing MiniSat's features one at a time show that the real difference in conflicts is exactly its restarts and clause minimization. Every answer is checked independently, including a DRAT proof for every unsatisfiable result.

1 The solver, and why its answers can be trusted

This report studies the performance of a CDCL SAT solver that I wrote from scratch in C++17, with no external dependencies. It implements the core of a modern CDCL solver [7, 8, 2]: two watched literals per clause, first-UIP clause learning, non-chronological backjumping, VSIDS branching with phase saving, clause deletion by LBD [1], and DRAT proof output. It does not yet have restarts, clause minimization, blocking literals, a heap for branching or arena clause storage; Section 9 uses the measurements to predict what each of these would bring.

A fast wrong answer is worthless, so no result in this report relies on the solver's own word. A satisfying assignment is easy to check: the test harness re-reads the formula from disk and checks the assignment clause by clause. An unsatisfiable answer has no such short certificate, so the solver writes a DRAT proof, which `drat-trim` [10] checks independently. The regression suite runs both checks on 1,010 instances, and a fuzzer compares 15,000 random formulas against exhaustive brute force, also with clause deletion forced to run early; there are no mismatches. An invariant checker guards the internal data structures, and it caught all six bugs I injected on purpose to test it.

Verification has a price (Table 1). Writing the proof costs 11–23% more cycles, and the overhead follows how much proof is written per unit of solving work (Spearman $\rho = 0.95$); proof logging never changes the search. Checking takes 0.7–1.5 times the solving time on the pigeonhole and random instances, but 9–27 times on the fullN family, which I have not investigated. `drat-trim` can also produce LRAT, the input format of the formally verified checker cake_lpr [9], so even the checker would not have to be trusted.

instance	conflicts	logging overhead	proof (MB)	check (s)	check / solve
`php9_8`	10360	+19.5%	1.2	0.12	1.5
`php10_9`	60416	+15.2%	9.9	1.13	1.3
`php11_10`	245609	+12.8%	54.0	7.12	1.3
`full15`	16384	+22.6%	1.0	0.57	9.2
`full16`	32768	+22.4%	2.3	2.89	19.7
`full17`	65536	+20.4%	5.1	9.90	27.3
`rand3_200_s1`	37502	+14.1%	4.1	0.33	0.9
`rand3_250_s2`	93978	+12.6%	12.8	0.99	0.8
`rand3_250_s3`	108163	+11.3%	14.7	1.18	0.7

Table 1: Cost of an independently checked UNSAT answer. Overhead is extra cycles with proof logging on; every proof was verified by `drat-trim`.

2 Method

All measurements come from one machine, an Apple M3 Pro whose performance cores run at 4.0 GHz. A small pointer-chasing program measured what a load costs on it: about 4 cycles from the L1 cache (128 KB), 15–36 cycles from L2 (16 MB) and 360–450 cycles from main memory, but only 4–8 cycles at any size when memory is read in order, because the processor then fetches the next data in advance. These numbers are the scale for every cost in this report.

Timings use the optimized build of the solver (`-O2`, assertions off), and profiles use the same build with debug symbols added. Instead of seconds I report processor cycles and instructions, read from `/usr/bin/time`, since cycles do not depend on the clock speed, and I keep the fastest of three runs. The solver also counts its own operations. These counts are identical on every run of the same input, so whenever a change should not affect the search I checked that they stayed identical. The counters cost 3–4% of the run time; my first version, which updated memory on every clause visit, cost 10%.

The solver is deterministic, but the same formula written differently (variables renamed, signs flipped, clauses reordered) can need twice as many conflicts, because the heuristics take a different path. I therefore compare configurations on shuffled copies of an instance, generated from fixed seeds, and in pairs: both configurations run on the same copy, so each copy's own difficulty cancels out in their ratio. A sign test and a Wilcoxon signed-rank test on the log ratios tell whether an effect is real, and a bootstrap 95% confidence interval for the median ratio gives its size.

Every experiment is a script that writes a CSV file recording the git commit, build flags, machine and command, and every figure, table and number in this report is generated from those files (Appendix A). Three limits apply throughout. The processor's hardware event counters were not available, so statements about caches and branches are inferences from several independent measurements, not direct counts. There is a single machine and all instances have at most 250 variables, so most of them fit in the L2 cache.

3 Where the time goes

I profiled with `sample`, the macOS profiler, which records the running instruction once per millisecond; a run of `php11_10` gives about 3,400 samples, so a share is accurate to about ± 1 percentage point. On that instance `propagate()` takes 93.2% of the run and `analyze()` 4.4%; everything else is below 1%. Cheap is not the same as unimportant, though: clause deletion takes almost no time itself, yet it made `php11_10` 4.4 times faster by keeping the clause database small.

Inside `propagate()`, about half of the whole run goes into searching a clause for a new literal to watch after one of its watched literals has become false (Figure 1), and one instruction alone receives 37% of all samples. A sample lands on the instruction the processor is *waiting at*, not on the one that caused the wait, an effect called skid. The hot instruction is a cheap load from L1, reachable only by a jump from the check that ends the search loop. That check depends on the current assignment and is hard to predict, and after a wrong guess the processor restarts exactly there. This points to branch mispredictions, a hypothesis I cannot confirm without hardware counters.

The random instance supports it. Its learnt clauses are shorter, so a search checks about 2 literals instead of 10: the loop body shrinks from 8% to 1.5% of the run, but the loop exit stays expensive, so the cost is paid per search, not per literal. Moving a watch, a store to an essentially random address, costs only 2.4%, because the processor waits for loads but not for stores.

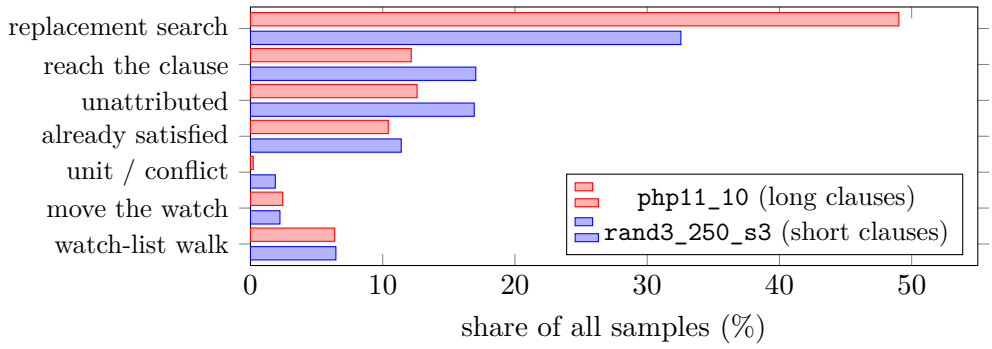


Figure 1: Share of all profile samples in each part of `propagate()`. Long learnt clauses (pigeonhole) make the search for a new watch dominant; short ones (random 3-SAT) shift weight to the fixed cost of each visit.

4 What a clause visit does

Looking at one clause that watches a literal that has just become false is a *clause visit*, the basic unit of work: `php11_10` performs 594 million of them. A visit is a *skip* if the clause is already satisfied, a *move* if another non-false literal takes over the watch, *unit* if the last literal is forced true, and a *conflict* if that literal is false as well. Counting all four gives two exact checks, since they must add up to the number of visits and the conflict visits must equal the conflicts; both hold on every instance.

instance	skip (%)	moved (%)	unit (%)	conflict (%)	literals per search
`php10_9`	70.6	28.6	0.81	0.072	9.5
`php11_10`	73.6	25.8	0.49	0.041	10.6
`rand3_250_s1`	61.9	32.9	5.12	0.092	2.2
`rand3_250_s3`	56.2	39.2	4.51	0.083	2.1
`full17`	97.1	2.8	0.06	0.057	8.1

Table 2: Outcome of every clause visit at the default deletion schedule.

Most visits are skips, 56–97% (Table 2), yet on `php11_10` the skip path takes only about 10% of the time: the quarter of visits that search for a new watch take half of the run. A search checks about 10.6 literals on `php11_10` but only 2.1 on `rand3_250_s3`, which explains why the two profiles in Figure 1 differ. Between 87% and 99% of all visits go to learnt clauses, so the deletion policy decides how much work propagation has to do.

One counter disproved an assumption. I expected the first check of each visit, whether the false literal sits in the first or the second watched position, to go either way about half the time. It is the first position only about 20% of the time, because the previous visit usually leaves it in the second. That branch is therefore easy to predict, and the time around it is more likely spent waiting for the clause's literals to arrive.

5 A cycle-level cost model

Can these counts explain the run time? If every kind of operation has a fixed cost, then

$$\text{cycles} \approx a \cdot \text{skips} + b \cdot \text{searches} + c \cdot \text{literals checked} + d \cdot \text{conflicts}, \tag{1}$$

where the searches are the visits that look for a new watch and the conflict term covers all per-conflict work: analysis, learning, backjumping and the next decision. I estimated a–d by least squares on the relative error over 39 runs: three families, each under four clause-deletion schedules, from 0.1 to 26 s. The schedules change the size of the clause database and with it the mix of operations, which is what lets the fit tell the four costs apart.

The result (Table 3) is 6.0 cycles per skip, 86.8 per search, 2.1 per literal checked and 4232 per conflict, and leaving out any single instance barely changes it. The model predicts the total cycles of a run with a median error of 2.5% (Figure 2). The costs are also physically plausible. A skip is cheaper than a single L2 access, which is only possible because independent visits overlap; a search, about 14 skips, leaves room for a misprediction plus several dependent loads; and a conflict, about 1.1 μs, matches the profile's share for `analyze()`. The processor agrees independently: the family dominated by cheap skips runs at 5.0–5.6 instructions per cycle, those dominated by searches at 1.6–2.3.

On runs it was not fitted on, the model is less reliable. Fitted on pigeonhole and fullN only, it underpredicts every random 3-SAT run by 15–21%, partly because both training families check 8–10 literals per search, so the fit cannot separate the cost per search from the cost per literal. Fitted on pigeonhole and random 3-SAT, it overpredicts fullN by 4–47%, and more so the more skips a run contains: a skip costs about 9 cycles on those two families but about 6 on fullN, whose visits are almost all predictable skips over clauses read largely in order (Section 6). The costs are accurate within the families they were fitted on, but they depend on the workload.

cycles per	estimate	leave-one-instance-out range
visit, clause already satisfied	6.0	[5.9, 6.2]
visit that searches for a new watch	86.8	[85.8, 88.7]
literal checked by a search	2.1	[1.5, 2.4]
conflict	4232	[3562, 4642]

Table 3: Per-operation costs fitted over 39 runs.

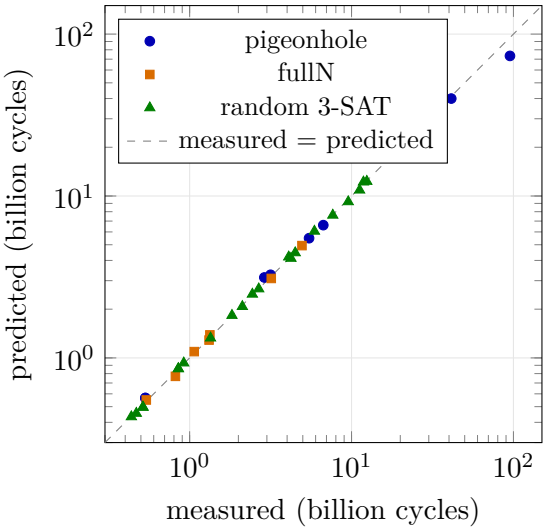


Figure 2: Measured against predicted cycles for every run (median error 2.5%). The outlier below the line is `php11_10` with clause deletion off, the only run whose working data far exceeds L2.

6 Memory and caches

Two results above point to memory: the worst prediction of the cost model was the run with the largest clause database, and skips are cheapest on fullN. The second curve in Figure 3 (left) explains the latter. Reading memory in order costs 4–8 cycles at any size, and although `full17` uses 31 MB, almost twice the L2, it creates its clauses one after another and mostly visits them in that order. Size alone does not make memory slow; size combined with a random access order does. Estimated from the data structures ,32 bytes per clause object, 4 bytes per literal in a separate block, 8 bytes of watch entries, the clause data of `php11_10` takes about 4 MB, well inside L2, so in the normal runs memory is not the bottleneck.

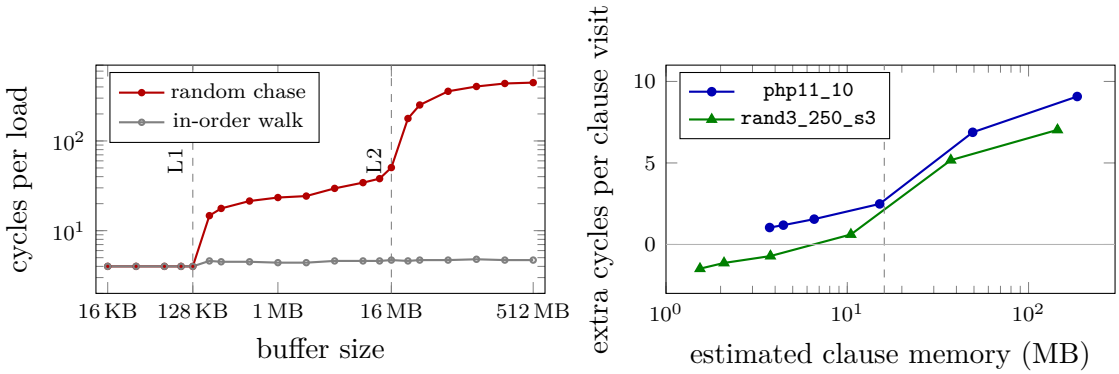


Figure 3: Left: load latency on this machine, for a random pointer chase and an in-order walk. Right: extra cycles per clause visit beyond the in-L2 cost model, as clause padding grows the footprint with the search held fixed. The dashed line marks the 16 MB L2.

To isolate memory, I added a setting, `SAT_CLAUSE_PAD`, that reserves unused space after every clause's literals. The literals are untouched, so the search is identical, while the footprint grows up to 20 times, from 11 to 226 MB on `php11_10`. Comparing each run with the cost model fitted only on in-L2 runs gives the extra cycles per visit caused by memory alone. They stay near zero up to an estimated 15 MB of clause data, rise between 15 and 40 MB, where the latency curve leaves L2, and reach about 9 cycles at the largest padding, making each visit 23% more expensive (Figure 3, right). That is far less than one main-memory access, about 450 cycles,

because independent visits overlap their misses.

region of `propagate()`	pad 0	pad 2048	change
replacement search	17.1	19.9	+2.8
unattributed	4.0	6.1	+2.1
already satisfied	3.6	5.6	+2.0
reach the clause	4.6	5.7	+1.1
watch-list walk	2.1	1.9	-0.1
move the watch	0.8	0.7	-0.2

Table 4: Cycles per clause visit by code region on `php11_10`, same search, unpadded and padded. Only regions that read clause literals grow.

A profile of the padded run shows where the cost lands (Table 4): only in code that reads the padded clause data. The watch-list walk and the watch move touch only memory the padding did not change, and they do not slow down, which makes them a control group inside the same measurement. Padding and a large clause database may leave the cache in different ways. Padding spreads the same hot data over more memory pages; one plausible explanation of its cost is that the processor runs out of address-translation entries (the TLB) rather than out of cache, but these measurements cannot tell TLB misses from cache misses. With clause deletion turned off, by contrast, `php11_10` keeps 166,675 learnt clauses and genuinely reads more data, at about 10 extra cycles per visit and 1.70 instructions per cycle. Without miss counts, four signals agree: the latency curve, the knee at the L2 size, the falling instructions per cycle, and the cost appearing only where padded data is read. Together they show that memory becomes more expensive beyond L2, but not which structure is responsible; a pointer chase that touches one cache line per page would measure the TLB's reach and separate the two.

7 Comparison with MiniSat

A single run made the solver look 12 times faster than MiniSat [2], with preprocessing off, on `php11_10`: 5.4 s against 65.6 s. Splitting the run time into conflicts and cost per conflict shows why. MiniSat needed 42 times more conflicts, because its learnt-clause limit is sized from the formula: it starts at about 200 clauses on this 561-clause instance, while this solver keeps up to 23,000. Each of MiniSat's conflicts was 3.5 times cheaper, but that does not show a faster engine, since with so few clauses there is far less to visit per conflict. Raising MiniSat's limit to 20,000 cut its conflicts tenfold but made each one 22 times more expensive: at 137 μs against this solver's 22 μs, it was then six times *slower* per conflict, while holding 23,000 to 74,000 clauses. Cost per conflict therefore mostly reflects the size of the clause database, not the efficiency of the engine. A fair engine comparison would need the cost per clause visit at an equal database size, which MiniSat does not report. Clause deletion is a trade-off: fewer clauses make each conflict cheaper, but there are more conflicts.

The comparison does not survive a luck control. Over 50 shuffled copies of `php9_8`, the number of conflicts varies about twofold for the same formula, and the result depends on the signs of the literals. MiniSat needs 1.51 times as many conflicts as this solver on copies with sign flips, but only 0.78 times as many on copies without (Figure 4). Only solvers without restarts are sensitive to the signs (Figure 5): with its restarts switched off, MiniSat shows the same twofold shift as this solver. Restarts protect a solver from an unlucky start [4, 6].

Removing MiniSat's features one at a time closes the gap. Without restarts, the ratio becomes 0.77 and 0.91; without restarts and clause minimization, it becomes 1.08 and 1.06, with confidence intervals that include 1. On this instance the whole difference in conflicts is these two features, and even the learnt clauses match: 24.4 and 26.3 literals for this solver, 24.9 and 26.8 for MiniSat

without minimization, whose clauses are about 19% shorter with it. The remaining differences, including LBD against activity for ranking learnt clauses, made no measurable difference here; how many clauses are kept mattered far more than which ones. Two further observations stay open. MiniSat's preprocessing increased its conflicts by about 1.3 times, but only with restarts on, and the original `php11_10` file is a lucky case for this solver: 245,609 conflicts, against a median of 389,126 over shuffled copies.

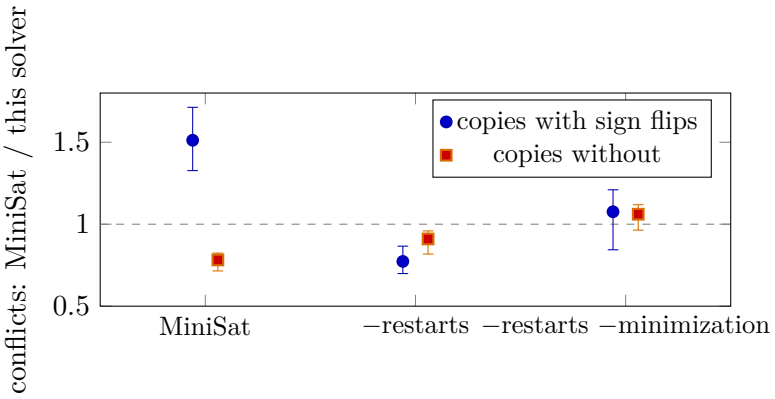


Figure 4: Conflicts of MiniSat (no preprocessing) divided by conflicts of this solver, median over 50 shuffled copies of `php9_8` with 95% bootstrap intervals. Removing restarts and clause minimization from MiniSat closes the gap for both kinds of copies.

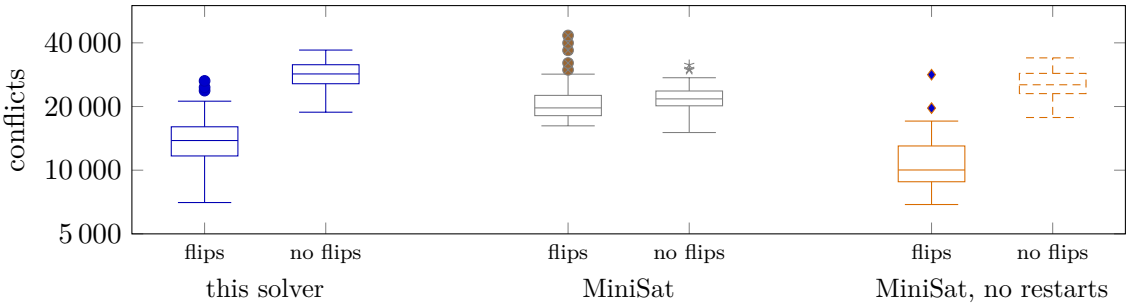


Figure 5: Conflicts over 50 shuffled copies of `php9_8`, with and without sign flips. Only the solvers without restarts are sensitive to the signs.

8 Which growth is the solver's fault?

Some growth is inherent. On pigeonhole formulas, the median number of conflicts over shuffled copies grows about 4.7 times per added pigeon, from 724 on `php7_6` to 389,126 on `php11_10`, while the formula itself grows only like $n^{3.2}$. No CDCL solver can avoid this: every resolution proof of the pigeonhole principle is exponentially long [5], and a CDCL run is such a proof. The fullN family, all 2^n clauses over n variables, is the opposite case. Removing any one clause makes it satisfiable, so every proof must use all 2^n clauses, and the solver needs exactly 2^{n-1} conflicts for every n from 13 to 17: work linear in the size of the input, matching that lower bound up to a constant.

9 What the measurements predict

Each planned feature can now be given a measured expectation before it is built (Table 5), so each one becomes a test of the model as well as an improvement.

next step	what the measurements predict
restarts	less sensitivity to variable order and signs (the twofold shift should shrink toward MiniSat's 1.1); not necessarily fewer conflicts on pigeonhole
clause minimization	learnt clauses about 19% shorter and 13–27% fewer conflicts (Section 7)
blocking literals	saves at most the cost of skips: 8–15% of cycles on pigeonhole, 4–19% on random 3-SAT, up to 87% on fullN; more once data leaves L2
cheaper search	the largest target, 62–82% of cycles on pigeonhole and random 3-SAT; resuming the scan where the last one stopped [3] would shorten it
VSIDS heap	no measurable effect at 250 variables; matters at 10^5
arena storage	removes one pointer indirection per visit, the cost measured in Section 6
binary DRAT	smaller proofs, less of the 11–23% logging overhead

Table 5: Measured expectations for the planned features.

10 Conclusion and open questions

The solver's run time is explained, to within a few percent, by four per-operation costs, and on a hard combinatorial instance its gap to MiniSat is exactly two missing features. The most useful lesson was about method: the first comparison suggested a twelvefold advantage, and only shuffled copies, paired statistics and removing features one at a time showed what that number was made of. Three questions remain open: why preprocessing hurts MiniSat only when restarts are on, which an experiment varying the restart interval could answer; why proof checking is so slow on fullN; and whether ranking clauses by LBD pays off on structured industrial instances, which the benchmark set here does not contain.

References

[1] Gilles Audemard and Laurent Simon. Predicting learnt clauses quality in modern SAT solvers. In *International Joint Conference on Artificial Intelligence (IJCAI 2009)*, pages 399–404, 2009.

[2] Niklas Eén and Niklas Sörensson. An extensible SAT-solver. In *Theory and Applications of Satisfiability Testing (SAT 2003)*, volume 2919 of *LNCS*, pages 502–518. Springer, 2003.

[3] Ian P. Gent. Optimal implementation of watched literals and more general techniques. *Journal of Artificial Intelligence Research*, 48:231–252, 2013.

[4] Carla P. Gomes, Bart Selman, Nuno Crato, and Henry Kautz. Heavy-tailed phenomena in satisfiability and constraint satisfaction problems. *Journal of Automated Reasoning*, 24(1–2):67–100, 2000.

[5] Armin Haken. The intractability of resolution. *Theoretical Computer Science*, 39:297–308, 1985.

[6] Michael Luby, Alistair Sinclair, and David Zuckerman. Optimal speedup of Las Vegas algorithms. *Information Processing Letters*, 47(4):173–180, 1993.

[7] João P. Marques-Silva and Karem A. Sakallah. GRASP: A search algorithm for propositional satisfiability. *IEEE Transactions on Computers*, 48(5):506–521, 1999.

[8] Matthew W. Moskewicz, Conor F. Madigan, Ying Zhao, Lintao Zhang, and Sharad Malik. Chaff: Engineering an efficient SAT solver. In *Design Automation Conference (DAC 2001)*, pages 530–535, 2001.

[9] Yong Kiam Tan, Marijn J. H. Heule, and Magnus O. Myreen. cake_lpr: Verified propagation redundancy checking in CakeML. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2021)*, volume 12652 of *LNCS*, pages 223–241. Springer, 2021.

[10] Nathan Wetzler, Marijn J. H. Heule, and Warren A. Hunt Jr. DRAT-trim: Efficient checking and trimming using expressive clausal proofs. In *Theory and Applications of Satisfiability Testing (SAT 2014)*, volume 8561 of *LNCS*, pages 422–429. Springer, 2014.

A Reproducing the results

All commands run from the repository root after `cd src && make release && make bench-gen`. Each script writes a CSV whose header records its configuration, and `docs/report/make_data.py` turns those files into every figure, table and number in this report.

result	command
regression and proofs	`cd src && make test`
fuzzing	`cd src && make fuzz && ./fuzz 15000`
profile (Sec. 3)	`sample <pid> 4 1 -file php.sample.txt`
cost model (Sec. 5)	`python3 tests/cost_model.py collect`
memory sweep (Sec. 6)	`python3 tests/memory_sweep.py run`
latency ladder	`clang -O2 tests/latency_ladder.c && ./a.out`
proof cost (Sec. 1)	`python3 tests/proof_cost.py -lrat`
proof complexity (Sec. 8)	`python3 tests/proof_complexity.py`
report data	`python3 docs/report/make_data.py`

The MiniSat comparison (Section 7) runs `tests/luck_test.py tests/bench/php9_8.cnf -seeds 1-50` six times: with and without `-no-flip`, each with default MiniSat, with `-minisat-flags "-rfirst=2147483647" -tag norestart`, and with `-minisat-flags "-rfirst=2147483647 -ccmin-mode=0" -tag norestart-nomin`.